

SESIÓN 13 Autorización por método con @PreAuthorize (JWT + Roles)

"La seguridad real está en el Service."

Bloque 1 — Por qué seguridad por método

Proteger por endpoint está bien, pero tiene límites:

- si mañana expones un endpoint nuevo y olvidas protegerlo → agujero
- si dos endpoints llaman al mismo caso de uso, duplicas reglas
- el negocio debe estar protegido "desde dentro"



Mensaje clave:

"El controller es una puerta. El Service es la caja fuerte."

Bloque 2 — Requisitos previos

JWT ya funcionando

Sistema de autenticación con tokens JWT implementado y operativo

Roles incluidos en UserDetails

Roles configurados en memoria o base de datos

@EnableMethodSecurity activo

Configuración habilitada en SecurityConfig

 Ya lo tenemos.

Bloque 3 — Conceptos clave

1. ¿Qué es un "authority" en Spring?

Spring trabaja con authorities (permisos). Los roles se representan normalmente como:

- `ROLE_ADMIN`
- `ROLE_USER`

Y por eso:

`hasRole("ADMIN")` equivale a `hasAuthority("ROLE_ADMIN")`

2. ¿Dónde se aplica `@PreAuthorize`?

En métodos de `@Service` (o `@Controller`, pero mejor service).

Ejemplos:

- `@PreAuthorize("hasRole('ADMIN')")`
- `@PreAuthorize("hasAnyRole('ADMIN','USER')")`
- `@PreAuthorize("isAuthenticated()")`

Bloque 4 — Ajuste importante en SecurityConfig

Cuando haces seguridad por método, puedes dejar el `SecurityFilterChain` más simple:

- deja pasar swagger + login
- el resto autenticado
- los roles se controlan en el service

 **config/SecurityConfig.java (recomendación)**

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
    .requestMatchers("/api/auth/**").permitAll()
    .anyRequest().authenticated()
)
```

 Esto reduce duplicación: no mezclas reglas por endpoint y por método.

Bloque 5 — Aplicar roles por método en BookService

Ejemplo de reglas realistas:

Cualquiera autenticado puede buscar/ver
Operaciones de lectura disponibles para todos los usuarios autenticados

Solo ADMIN puede crear, actualizar y borrar
Operaciones de escritura restringidas a administradores

service/BookService.java (añadir anotaciones)

```
package com.cladsb1.books.service;

import com.cladsb1.books.dto.request.BookFilterDTO;
import com.cladsb1.books.dto.request.BookRequestDTO;
import com.cladsb1.books.dto.response.BookResponseDTO;
import com.cladsb1.books.dto.response.PageResponseDTO;
import com.cladsb1.books.entity.Book;
import com.cladsb1.books.entity.Category;
import com.cladsb1.books.mapper.BookMapper;
import com.cladsb1.books.repository.BookRepository;
import com.cladsb1.books.repository.CategoryRepository;
import com.cladsb1.books.spec.BookSpecifications;
import org.springframework.data.domain.*;
import org.springframework.data.jpa.domain.Specification;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.Optional;
```

BookService con @PreAuthorize

```
@Service
public class BookService {

    private final BookRepository bookRepository;
    private final CategoryRepository categoryRepository;
    private final BookMapper bookMapper;

    public BookService(BookRepository bookRepository,
        CategoryRepository categoryRepository,
        BookMapper bookMapper) {
        this.bookRepository = bookRepository;
        this.categoryRepository = categoryRepository;
        this.bookMapper = bookMapper;
    }

    //
```



Bloque 6 — Controller se queda "neutro"

El controller no necesita saber roles:

```
@GetMapping("/search")
public ResponseEntity search(..., Pageable pageable) {
    return ResponseEntity.ok(bookService.search(
        new BookFilterDTO(title, author, category), pageable));
}
```

Si entra un USER intentando POST:



Bloque 7 — Manejo de errores de seguridad

Para devolver errores con formato consistente (como tu ApiError), añadimos handlers:

📁 **error/GlobalExceptionHandler.java** (añadir)

```
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.*;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.core.AuthenticationException;
import org.springframework.web.bind.annotation.ExceptionHandler;

import java.time.Instant;
import java.util.Map;

@ExceptionHandler(AccessDeniedException.class)
public ResponseEntity handleAccessDenied(AccessDeniedException ex,
                                         HttpServletRequest request) {
    ApiError body = new ApiError(
        Instant.now(),
        HttpStatus.FORBIDDEN.value(),
        "Forbidden",
        request.getRequestURI(),
        Map.of()
    );
    return ResponseEntity.status(HttpStatus.FORBIDDEN).body(body);
}

/**
 * Nota: AuthenticationException suele dispararse antes (filtros),
 * pero este handler ayuda en algunos flujos.
 */

@ExceptionHandler(AuthenticationException.class)
public ResponseEntity handleAuth(AuthenticationException ex,
                                 HttpServletRequest request) {
    ApiError body = new ApiError(
        Instant.now(),
        HttpStatus.UNAUTHORIZED.value(),
        "Unauthorized",
        request.getRequestURI(),
        Map.of()
    );
    return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(body);
}
```

Mensaje clave:

"401 = no autenticado. 403 = autenticado sin permisos."

Bloque 8 — Pruebas en Swagger

Login con user/user123 → token USER

Authorize Bearer token

Probar:

- GET /api/books/search →  200
- POST /api/books →  403

Login con admin/admin123 → token ADMIN

Probar:

- POST /api/books →  201

Bloque 9 — Variantes útiles de @PreAuthorize

Solo autenticados:

```
@PreAuthorize("isAuthenticated()")
```

Admin o User:

```
@PreAuthorize("hasAnyRole('ADMIN','USER')")
```

Authority exacta:

```
@PreAuthorize("hasAuthority('ROLE_ADMIN')")
```

 **Cierre**

"Proteger rutas está bien. Proteger métodos es robusto."